Hospitality App to Kubernetes Report

Deploy Hospitality App to Kubernetes

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Dhruv Dwivedi	EN22CS301333
2.	Diya Jain	EN22CS301352
3.	Dishant Rathore	EN22CS301339
4.	Chitralekha	EN22CS301304
5.	Bhavesh Thakur	EN22CS301265

Group Name: Group 04D3

Project Number: DO-28

Industry Mentor Name: Prof. Suraj Nayak

University Mentor Name: Prof. Shyam Patel

Academic Year: 2022 - 2026

1. Problem Statement & Objectives

1. Problem Statement
2. Project Objectives
3. Scope of the Project

2.Proposed Solution

1. Key features
2. Overall Architecture / Workflow
3. Tools & Technologies Used

3.Results & Output

1. Screenshots / outputs
2. Reports / dashboards / models
3. Key outcomes

4.Conclusion

5.Future Scope & Enhancements

1. Problem Statement

In modern hospitality platforms, maintaining high availability, scalability, and seamless deployment is essential for delivering uninterrupted services to users. Traditional deployment methods often lead to downtime during updates, inefficient resource utilization, and difficulty in managing multiple services.

The problem statement of this project is to deploy a Hospitality Application using Kubernetes (Minikube) by containerizing the application with Docker and managing deployments through Kubernetes manifests. The goal is to ensure smooth deployment, service exposure, scalability, and zero-downtime rolling updates while maintaining system reliability.

2. Project Objectives

The major objectives of this project are:

- To containerize the Hospitality Application using Docker
- To manage source code and version control using Git and GitHub
- To automate build and deployment workflows using GitHub Actions / Jenkins
- To deploy the application on a Kubernetes cluster using Minikube
- To create Deployment and Service YAML files for orchestration
- To implement Rolling Updates for zero-downtime deployment
- To improve scalability, maintainability, and fault tolerance of the application

3. Scope of the Project

This project focuses on DevOps practices for deploying a hospitality platform application in a production-like environment using Kubernetes.

The scope includes:

- Docker containerization
- CI/CD workflow integration
- Kubernetes deployment and service management
- Rolling updates and version management
- Cluster monitoring and pod management
- Service availability without downtime

This project does not focus on application development but emphasizes deployment, orchestration, and infrastructure management.

Proposed Solution

1. Key Features

- Dockerized Hospitality Application
- Kubernetes Deployment Management
- Kubernetes Service Exposure
- GitHub-based Source Control
- CI/CD Pipeline using GitHub Actions
- Rolling Updates without downtime
- Pod Scaling and Replica Management
- Container Monitoring and Status Tracking
- Minikube Local Kubernetes Cluster Setup

2. Overall Architecture / Workflow

Step 1: Application Development

The Hospitality Application was developed using Python and Streamlit for the frontend interface. The application includes modules like Trip Planner and Hospitality Creator.

Step 2: Version Control using Git

The complete project source code was maintained using Git and pushed to GitHub for collaboration and version tracking.

Step 3: Docker Containerization

A Dockerfile was created to package the application along with all dependencies into a Docker image.

Step 4: Docker Hub Push

The Docker image was built and pushed to Docker Hub for centralized image management.

Step 5: Kubernetes Deployment

Using Minikube, a local Kubernetes cluster was created. Deployment YAML and Service YAML files were written to define:

- Application deployment
- Replica management
- Port exposure
- Service accessibility

Step 6: Rolling Updates

Kubernetes rolling update strategy was implemented to update container versions without causing service downtime.

Step 7: Monitoring

Pods, deployments, and services were continuously monitored using kubectl commands.

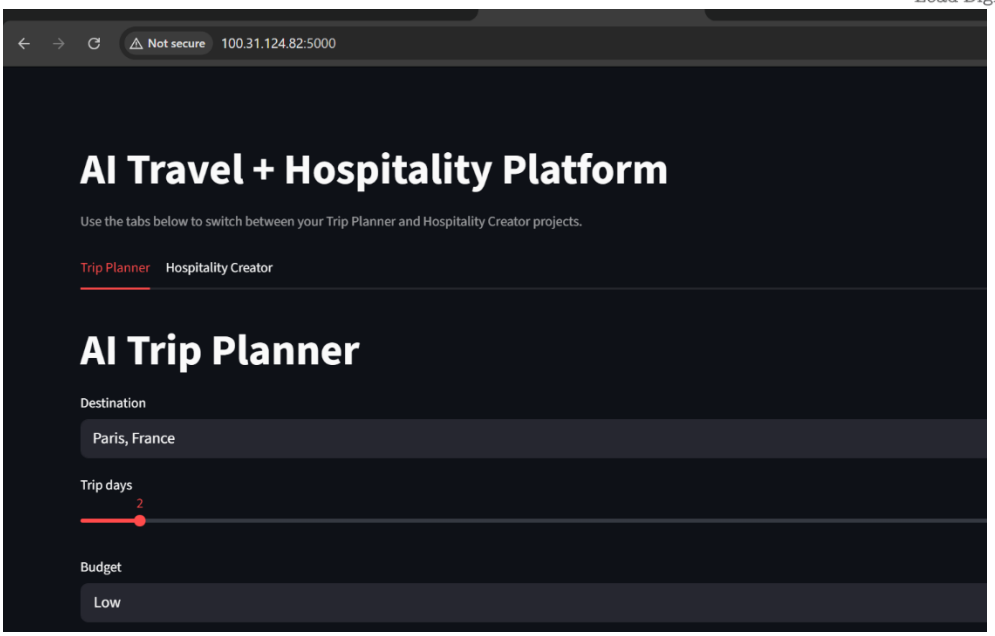
3. Tools & Technologies Used

Tool / Technology	Purpose
Git	Version Control
GitHub	Source Code Hosting
Docker	Application Containerization
Docker Hub	Image Repository
Kubernetes	Container Orchestration
Minikube	Local Kubernetes Cluster
GitHub Actions	CI/CD Automation
Streamlit	Frontend Application
Python	Backend Development
YAML	Kubernetes Configuration Files
AWS EC2	Cloud Deployment Testing

Results & Output

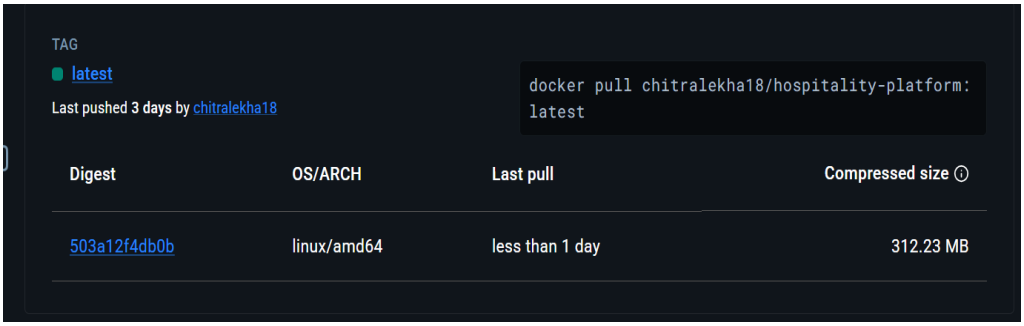
1. Outputs

The following outputs were successfully achieved:



Docker Output

- Docker image built successfully



- Docker container running properly
- Application accessible through container port mapping

Kubernetes Output

- Pods created successfully

```
ubuntu@ip-172-31-6-188:~/Hospitality_platform$ kubectl get nodes
NAME       STATUS   ROLES           AGE    VERSION
minikube   Ready    control-plane   6d6h   v1.35.1
ubuntu@ip-172-31-6-188:~/Hospitality_platform$
```

- Deployment created using deployment.yaml
- Service created using service.yaml
- Application accessible using Minikube service

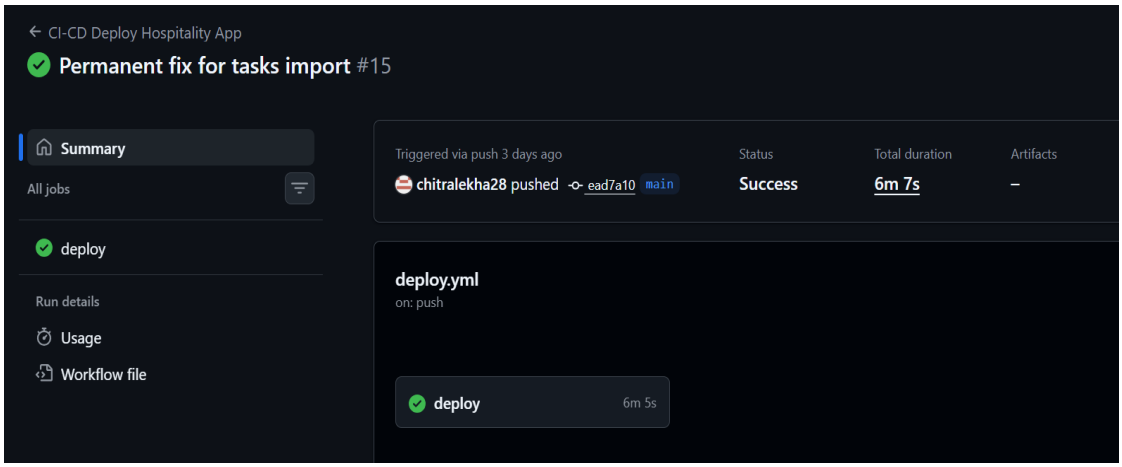
Rolling Update Output

- New image version deployed successfully
- Old pod replaced automatically
- No service downtime observed during updates

```
ubuntu@ip-172-31-6-188:~/Hospitality_platform$ kubectl rollout status deployment hospitality-app
deployment "hospitality-app" successfully rolled out
ubuntu@ip-172-31-6-188:~/Hospitality_platform$
```

CI/CD Output

- GitHub Actions workflow executed successfully
- Automated deployment process improved efficiency



2. Reports / Dashboards / Models

Deployment Report

The deployment report showed:

- Successful pod creation
- Running container instances
- Replica management
- Stable service exposure

Monitoring Dashboard

Using kubectl commands:

- kubectl get pods
- kubectl get deployments

- kubectl get services
- kubectl rollout status deployment

```
ubuntu@ip-172-31-6-188:~/Hospitality_platform$ kubectl get nodes
NAME       STATUS   ROLES           AGE    VERSION
minikube   Ready    control-plane   6d6h   v1.35.1
ubuntu@ip-172-31-6-188:~/Hospitality_platform$ kubectl get pods
NAME                                  READY   STATUS    RESTARTS      AGE
hospitality-app-75f8dbb7f9-chppj      1/1     Running   2 (25h ago)   2d16h
hospitality-app-75f8dbb7f9-jmkxl      1/1     Running   2 (25h ago)   2d16h
hospitality-app-75f8dbb7f9-w969k      1/1     Running   2 (25h ago)   2d16h
ubuntu@ip-172-31-6-188:~/Hospitality_platform$ kubectl get deployments
NAME              READY   UP-TO-DATE   AVAILABLE   AGE
hospitality-app   3/3     3            3           6d6h
ubuntu@ip-172-31-6-188:~/Hospitality_platform$ kubectl get services
NAME                  TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
hospitality-service   NodePort    10.96.194.197   <none>        5000:30007/TCP   6d5h
kubernetes            ClusterIP   10.96.0.1       <none>        443/TCP          6d6h
ubuntu@ip-172-31-6-188:~/Hospitality_platform$
```

System performance and deployment health were tracked successfully.

3. Key Outcomes

- Successful Kubernetes deployment of Hospitality App
- Zero-downtime rolling updates achieved
- Improved deployment automation using CI/CD
- Better resource utilization through container orchestration
- Scalable and fault-tolerant deployment environment
- Practical understanding of real-world DevOps workflow

Conclusion

This project successfully demonstrated the deployment of a Hospitality Application using Kubernetes with Docker-based containerization and CI/CD automation. By integrating Git, Docker, GitHub Actions, Minikube, and Kubernetes, the team achieved efficient deployment management and zero-downtime rolling updates.

The project provided strong practical exposure to DevOps concepts such as containerization, orchestration, deployment automation, scalability, and fault tolerance. It also helped us understand real-world deployment challenges and solutions used in modern software industries.

This project significantly enhanced our knowledge of Kubernetes-based application deployment and prepared us for handling enterprise-level deployment systems.

Future Scope & Enhancements

The future enhancements for this project include:

- Deployment on cloud-managed Kubernetes services like AWS EKS, Azure AKS, or Google GKE
- Auto-scaling using Horizontal Pod Autoscaler (HPA)
- Monitoring integration using Prometheus and Grafana
- Logging system using ELK Stack
- Advanced CI/CD pipeline using Jenkins
- Helm chart implementation for simplified Kubernetes deployment
- Multi-environment deployment support (Dev, Test, Production)
- Security enhancement using Kubernetes Secrets and ConfigMaps
- Load balancing and ingress controller implementation

These improvements can make the system more production-ready and enterprise scalable.